

Embedded Systems II

EEE3096/5S



FINAL EXAM 19 November 2025 3 hours

*Examination Prepared by: Simon Winberg & Best Nkhumeleni
Last Modified: 18-Oct-2025*

MEMO!!

REGULATIONS

Dear Students,

Please note this is a **closed-book exam**. Provide your answers to the questions in the provided UCT examination booklet. You are recommended to detach the cheat sheets that are on the last pages of the exam paper. The exam is printed double sided. You are recommended to first scan through the questions quickly before starting, so that you can plan your strategy for answering the questions. If you are caught cheating, you will be referred to University Court for expulsion procedures. Make sure to **put your student name and student number** on your answer booklets. Please indicated at least your student number on the answer booklets with the **course code EEE3096S or EEE3095S and a title ES2 Final Exam**.

DO NOT TURN OVER UNTIL YOU ARE TOLD TO

Exam Structure

Marked out of 100 marks. 180 minutes (30min of which incorporates reading time for questions).

RULES

-
- **Answer all questions; no questions are optional and will lose marks if no attempt is made to answer**
 - Make sure that you cross out material you do not want marked. Your first attempt at any question will be marked if two answers are found.
 - Use one of the answer book to plan the facts for your written replies to questions, so that you produce carefully constructed responses (then cross out rough work to avoid the possibility of the rough attempt being marked).
 - Answer all questions, and note that the time for each question relates to the marks allocated (which implies there are a few minutes to allow you to think and shuffle papers etc).



Structure of exam:

C: Design Questions / problem-based scenarios to test your mettle

B: Short Answers

A: Multiple Choice

NOTE: detachable multiple-choice answer sheet... detach it, put your name on it

SECTION A:

MULTIPLE CHOICE AND SOME EASYPEASY QUESTIONS [5 x 8 = 40 marks]

Circle the letter a - d to select your choice of answer
(select only one answer option unless explicitly stated otherwise)

Question A1. [5 marks]

In SPI, when moving from idle to start, which is controlled by the master, which of the following needs to occur before the slave will start receiving the data so that the master can start sending the data?

Choose one of the options below...

- a) The Master Input Slave Output line must be held LOW
- b) The slave select line must change from LOW to HIGH
- c) The slave select line must change from HIGH to LOW ← answer
- d) The Master Output Slave Input line must be held HIGH

Question A2. [5 marks]

There are various classifications for real-time systems. Which one of the following options best defines the characteristics of an asynchronous real-time system?

Select *the most correct* option below, indicating the result:

- a) A system that meets (at least one) specific hard real-time deadline that repeats at a specific interval.
- b) It is a system for which any one of its operations has to be completed within a specific time bound. ← answer
- c) A system with a set of tasks that executes repeatedly at a specific period.
- d) The term refers to a real-time system that uses polling (as opposed to e.g., interrupts).

Question A3. [5 marks]

What is the hardware complexity of the standard Flash ADC design? Where n refers to the number of codes that the ADC has.

Select *only one* option below, indicating the result:

- a) $O(\log(n))$
- b) $O(1)$
- c) $O(n)$ ← answer
- d) $O(n\log(n))$

Question A4. [5 marks]

What is the fastest communication protocol of the protocols mentioned in these options?

- a) I2C
- b) RS232
- c) SPI ← answer
- d) RS485

Question A5. [5 marks]

Consider a digital video playback system, such as a streaming app on a smart TV. The system's deadline is to decode and display each video frame before the next one is due (e.g., within 40ms for a 25 FPS video). If the processor is temporarily busy and fails to meet the deadline for a single frame, the system might drop that frame, causing a momentary stutter. Nevertheless, the video continues to play, and the overall viewing experience is only slightly degraded. Based on this behavior, how would you classify this video playback system? (select only one option below).

- a) Hard real-time
- b) Firm real-time
- c) Aperiodic real-time
- d) Soft real-time ← answer

Memo:

Soft real-time. The key characteristic is that missing a deadline does not lead to a catastrophic failure of the system. Instead, it results in a graceful degradation of performance or quality of service (the video stutters but continues). The utility of the task (displaying a frame) diminishes after the deadline, which is the definition of a soft real-time system.

Considering other options and why not a suitable answer...

- a) It is not Hard real-time because a missed deadline is not a critical system failure.
- b) While it has elements of Firm real-time (a late frame is useless and is dropped), the overall system characteristic is one of performance degradation, which is best described as soft.
- c) Aperiodic describes the arrival pattern of tasks, not the nature of their deadlines

Question A6. [5 marks]

According to the Brisbane definition, an embedded system is described as:

- a) a system that integrates computation with physical dynamics
- b) a task-specific computer, which is built into a larger system for the purpose of controlling and monitoring the larger system ← answer
- c) An information processing system embedded into a larger product
- d) a computer system in which a mechanism is controlled or monitored by computer-based algorithms

Question A7. [5 marks]

Consider the following ARM assembly function. What value will be in register r0 when the function returns, if it is called with r0 = 5. Select *only one* option below to indicate your answer.

```
my_func:
    MOV r1, #0          // set counter r1 to 0
    MOV r2, #1          // use r2 as a bit-mask, starting at bit 0
loop:
    CMP r0, #0          // check if the input value is zero
    BEQ end             // if zero then exit
    AND r3, r0, r2      // check if the current bit is set
    CMP r3, r2          // compare to the mask
    ADDEQ r1, r1, #1     // if equal (bit was set), increment counter
    LSR r0, r0, #1      // logical shift right to check next bit
```

```
        B loop
end:
    MOV r0, r1      // Move the final count to r0 for return
    BX lr
```

- a) 1
- b) 2 ← answer
- c) 3
- d) 5

Question A8. [5 marks]

What compiler optimization technique involves replacing a function call with the actual code of the function itself where the call is made? Select *only one* answer option below.

- a) Loop unrolling
- b) Function inlining ← answer
- c) Dead code elimination
- d) Constant folding

SECTION B:

Embedded communications, DACs and ADC's [30 marks]

Question B1. [10 marks]

This question comprises two sub-questions, B1.1 and B1.2.

B1.1. Distinguish between synchronous and asynchronous serial communication. Provide one example protocol for each and explain why it fits that category. [5 marks]

Memo: Synchronous serial communication uses a shared clock signal between the transmitter and receiver to synchronize data transfer. All data bits are transferred on the edge of this shared clock. This allows for higher data rates.

Example: SPI (Serial Peripheral Interface). SPI uses a dedicated clock line (SCLK) that is controlled by the master device to tell the slave device exactly when to sample the data line (MOSI/MISO).

Asynchronous serial communication does not use a shared clock signal. Instead, timing is agreed upon beforehand (baud rate), and synchronization is achieved for each data packet using start and stop bits.

Example: UART (used in RS232). A UART transmission begins with a start bit, which signals the receiver to start its own internal clock. The receiver then samples the data bits at the pre-agreed baud rate until it sees a stop bit, at which point it waits for the next start bit.

B1.2. In an embedded C development toolchain, what are the distinct roles of the **linker** and the **loader**? Briefly explain what each one is to clarify how they differ. [5 marks]

Memo:

Linker: The linker's job is to take one or more object files (the output of the compiler/assembler) and combine them into a single executable file. Its key responsibilities are:

Symbol Resolution: It resolves external references, where code in one file calls a function or accesses a variable defined in another file, by replacing symbolic addresses with actual memory addresses.

Relocation: It arranges the code and data from all object files into memory sections (e.g., .text, .data, .bss) and adjusts symbolic references to their final memory locations.

Loader: The loader is the part of the operating system or boot sequence that is responsible for loading the executable file generated by the linker into the system's memory and preparing it for execution. Its key responsibilities are:

Loading: It copies the code and data sections from the executable file on storage (e.g., Flash) into RAM.

Initialization: It may perform final setup tasks, such as initializing the stack pointer and jumping to the program's entry point (e.g., `_start` or `main`) to begin execution.

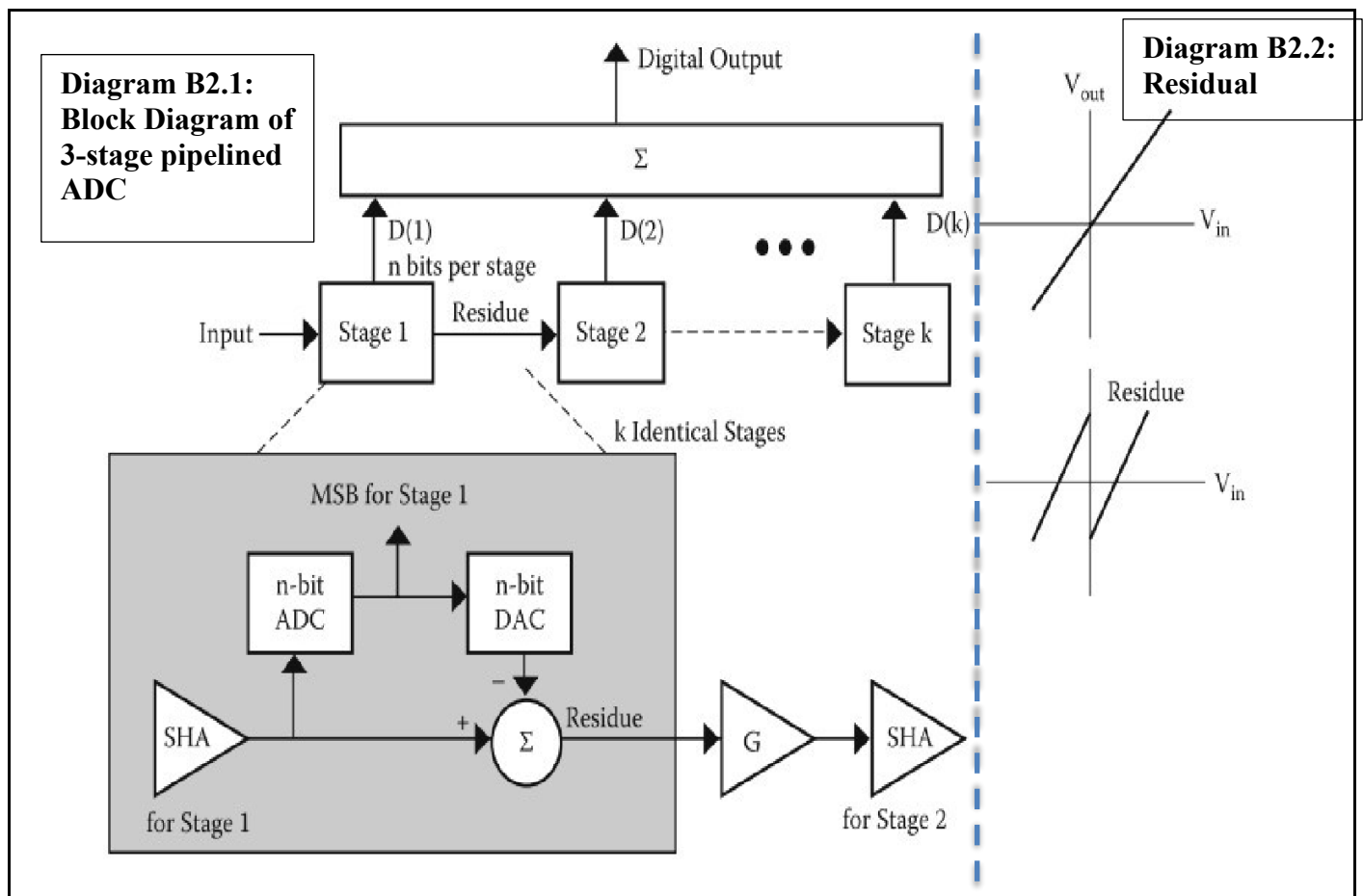
Question B2. [10 marks]

Consider a multi-stage Pipelined ADC. It is something of a design compromise compared to other types of ADCs that we covered in class. Provide the following two diagrams related to a Pipeline ADC. You can do this in pencil or pen but do provide a clear title for each diagram to assist the marker. Diagrams to do:

Diagram B2.1: Draw the block diagram of a 3-stage Pipeline ADC. For one of the stages, you must draw and label its internal components: a Sample-and-Hold (S/H), a sub-ADC, a sub-DAC, a subtractor, and an amplifier. [7 marks]

Diagram B2.2: Draw a simple diagram illustrating the residue plot (Residue Voltage vs. Input Voltage) for a single ideal stage. [3 marks]

B2 Memo:



Question B3. [5 x 2 marks each = 10 marks]

At last, the second most desired moment you've been awaiting: the TRUE / FALSE questions!
(The most desired moment is probably finishing the exam).

Use Multiple Choice answer page answer these questions, putting your answers in the Section B3 answer table. Answer the following statements as **true** to indicate that the statement is correct, or **false** to indicate the statement is not correct. Answer only yes or no for each statement, selecting both will get a mark of 0.

B3.1. A Digital-to-Analog Converter (DAC) created by applying a low-pass filter to a PWM signal from a GPIO pin is generally less noisy than a dedicated DAC ICs. (← Answer FALSE)

True / False ?

B3.2. The standard SPI (Serial Peripheral Interface) protocol is designed to support multiple masters on the same bus without requiring additional logic. (← Answer FALSE)

True / False ?

B3.3. PWM resolution is determined by the power draw of the DAC (← Answer FALSE)

True / False ?

B3.4. The Nyquist-Shannon sampling theorem states that to perfectly reconstruct a signal, the sampling rate must be strictly greater than twice the signal's highest frequency component. (← Answer TRUE)

True / False ?

B3.5. Aliasing only occurs when the sampling rate is less than twice the sampled signal's frequency or largest harmonic. (← Answer TRUE)

True / False ?

SECTION C:

TESTING YOUR METTLE QUESTIONS [30 marks]

The Following Scenario Relates to Questions C.1 and C.2 that follow

Take a bit of time to read this scenario and design concept; the time for the exam is planned on you taking about 15 minutes reading and making sense of the scenario, and then answer the questions that follow.

Scenario: “Auto-Vent Greenhouse Controller”

We are having a change from the PacePod in this activity... but you could imagine that the PacePod dev company you were working at has expanded their line of products, now getting into Greenhouse Controllers. Consider that you have been tasked with designing a simple, automated ventilation controller for a small greenhouse. The system's purpose is to prevent overheating by controlling a ventilation fan. And they like to do things cheap... or rather have a very economical version that is accessible to a wide market. Here follows details of the system parts and the vent operation that you need to take further in proposing design solutions for this product.



Figure 1: Auto-vent Controller

Auto-Vent System Components & Operation

Inputs:

- temp_sensor: An analogue sensor that measures the internal temperature. An ADC is attached to this to provide a digital value that a microcontroller can monitor.
- master_switch: A physical ON/OFF switch for the system.

Outputs:

- fan_motor: An actuator (such as a FET) to turn the fan ON or OFF.
- status_led: An LED to indicate the system's state.

Operational Logic

The system needs to perform these operations:

1. When the system is first connected to powered, but with the master_switch in the OFF position, it is in an OFF state. The fan is off and LED is off. This master switch could be implemented as a small slide switch that doesn't actually channel power to the device but just indicate the mode as on or off.
2. When the master_switch is turned ON, the system transitions to an IDLE state. The fan remains off, but the status_led should blink slowly (e.g., 1 second on, 1 second off) to show it's alive.
3. From the IDLE state, the system continuously monitors the temperature.
4. If the temperature rises above a high threshold ($TEMP_HIGH=30^{\circ}C$), the system should enter a COOLING state. In this state, the fan_motor is turned ON and the status_led turns solid ON (no longer blinking, showing the device is doing something or waiting for more favorable condition).
5. While in the COOLING state, the system continues to monitor the temperature. If the temperature drops below a low threshold ($TEMP_LOW=25^{\circ}C$), it should return to the IDLE state (fan turns off, LED resumes slow blinking). The use of two thresholds is aimed, as you likely suspect, to prevent the fan from rapidly switching on and off (i.e., implements 'hysteresis' to change over a time).
6. At any point, if the master_switch is turned OFF, the system must immediately return to the OFF state, turning off the fan and the LED.

Question C1. [15 marks]

C1.1. Based on the scenario given above, identify and list the distinct operational states of the Auto-Vent system. For each state, describe the status of the fan_motor and status_led outputs. [4 marks]

Memo:

There are three distinct operational states:

OFF: fan_motor is OFF, status_led is OFF.

IDLE: fan_motor is OFF, status_led is BLINKING (slowly).

COOLING: fan_motor is ON, status_led is SOLID ON.

C1.2. Draw a complete state transition diagram (state chart) that models the behavior of the Auto-Vent system. Your diagram must clearly show all states, the conditions that cause transitions between states, and any actions that occur on those transitions. [11 marks]

Memo:

[A diagram would be drawn here. The description below matches the expected diagram.]

The state diagram should have three circles representing the states: OFF, IDLE, and COOLING.

- Initial State: An arrow from a filled circle points to the OFF state, representing the initial state on power-up.
- Transitions from OFF:
 - An arrow from OFF to IDLE.
 - Condition: master_switch == ON
 - Action: start_blinking_led()
- Transitions from IDLE:
 - An arrow from IDLE to COOLING.
 - Condition: temperature > TEMP_HIGH
 - Action: fan_motor = ON, status_led = SOLID_ON
 - An arrow from IDLE to OFF.
 - Condition: master_switch == OFF
 - Action: fan_motor = OFF, status_led = OFF
- Transitions from COOLING:
 - An arrow from COOLING to IDLE.
 - Condition: temperature < TEMP_LOW
 - Action: fan_motor = OFF, start_blinking_led()
 - An arrow from COOLING to OFF.
 - Condition: master_switch == OFF
 - Action: fan_motor = OFF, status_led = OFF

(Marks are awarded for correctly identifying all states, all transitions, and all conditions/actions. Clarity of the diagram is also important).

Question C2. [15 marks]

C2.1. Write a C function, `run_controller_tick()`, that implements one iteration of the controller's logic based on your state machine from C1 above. You may assume the current state is stored in a global variable `current_state`. Use the following function prototypes for interacting with hardware:

[10 marks]

Note: if needed you can add comments briefly motivate aspects in your C function that may be different your state machine response – i.e. your answer to question C1 and C2 doesn't have to be aligned in consideration that you might not answer the one question immediately after answering the other.

Functions provided for interacting with hardware

```
// Reads the master switch status
int read_master_switch(); // Returns 1 for ON, 0 for OFF

// Reads the temperature sensor
int read_temperature(); // Returns temperature in degrees C

// Controls the fan and LED
void set_fan(int state); // 1 for ON, 0 for OFF
void set_led(int state); // 1 for SOLID_ON, 0 for OFF, 2 for BLINKING
```

Memo:

```
// Define states for clarity
typedef enum {
    STATE_OFF,
    STATE_IDLE,
    STATE_COOLING
} controller_state_t;

// Assume this is a global variable
controller_state_t current_state = STATE_OFF;

// Constants for temperature thresholds
#define TEMP_HIGH 30
#define TEMP_LOW 25

void run_controller_tick() {
    // First, check the master switch as it's the highest priority input
    if (read_master_switch() == 0) {
        current_state = STATE_OFF;
    }

    // Use a switch statement to implement the FSM logic
    switch (current_state) {
        case STATE_OFF:
            set_fan(0);
            set_led(0); // LED OFF
            if (read_master_switch() == 1) {
                current_state = STATE_IDLE;
            }
            break;
```

```

    case STATE_IDLE:
        set_fan(0);
        set_led(2); // LED BLINKING
        if (read_temperature() > TEMP_HIGH) {
            current_state = STATE_COOLING;
        }
        // The master switch check at the top handles the transition to OFF
        break;

    case STATE_COOLING:
        set_fan(1);
        set_led(1); // LED SOLID ON
        if (read_temperature() < TEMP_LOW) {
            current_state = STATE_IDLE;
        }
        // The master switch check at the top handles the transition to OFF
        break;
}
}

```

C2.2. The temperature sensor is analogue. Discuss the ADC resolution (in bits) you would choose for this application. Justify your choice by considering the trade-offs between precision, cost, and complexity for a simple greenhouse controller. It is expected that students would be aware of the different types of ADCs, such as options that take less logic and could be lower cost or need less power.

[5 marks]

Memo:

For a simple greenhouse controller, high precision is not required. We only need to reliably distinguish between temperatures around the 25°C and 30°C thresholds. Students are specifically asked to discuss ADC, so suggesting a 1-bit below/above approach that could e.g. involve a simple comparator circuit instead of an ADC is not a suitable solution, and such a design would not (easily) accommodate changing parameters. Thus, an answer that suggesting such a solution or avoiding ADCs would at best get around 3 marks.

A low-resolution ADC (e.g., 8-bit) would likely be sufficient. An 8-bit ADC provides $2^8 = 256$ distinct levels. If the sensor's range is 0°C to 100°C, the precision would be $100/256 \approx 0.4^\circ\text{C}$ per step. This is more than enough to differentiate between 25°C and 30°C and implement the hysteresis gap effectively.

A mid-resolution ADC (e.g., 10-bit or 12-bit), commonly found on microcontrollers, would provide even better precision (1024 or 4096 levels). For example, a 10-bit ADC over a 100°C range gives $\sim 0.1^\circ\text{C}$ precision. While this is overkill for simply controlling a fan, it might be chosen if it's a standard feature on the selected microcontroller and adds no extra cost. It would also allow for more accurate data logging if that feature were added later. This sort of answer especially if suggesting leveraging an ADC that is anyway incorporated in a lowcost ADC would certainly get towards top (5) marks.

A high-resolution ADC (>16-bit) would be unnecessary and expensive. The added precision provides no benefit for this application and increases cost and complexity.

Main points: An 8-bit or 10-bit ADC is a good choice. The 8-bit option is fully sufficient and minimizes cost, while a 10-bit ADC is a common and acceptable alternative if available on the chosen microcontroller and available at no extra cost.

END OF EXAMINATION

Appendix A: Verilog Cheat sheet

Numbers and constants

Example: 4-bit constant 10 in binary, hex and in decimal: 4'b1010 == 4'ha -- 4'd10

(numbers are unsigned by default)

Concatenation of bits using {}

4'b1011 == {2'b10, 2'b11}

Constants are declared using parameter:

parameter myparam = 51

Operators

Arithmetic: and (+), subtract (-), multiply (*), divide (/) and modulus (%) all provided.

Shift: left (<<), shift right (>>)

Relational ops: equal (==), not-equal (!=), less-than (<), less-than or equal (<=), greater-than (>), greater-than or equal (>=).

Bitwise ops: and (&), or (|), xor (^), not (~)

Logical operators: and (&&) or (||) not (!) note that these work as in C, e.g. (2 && 1) == 1

Bit reduction operators: [n] n=bit to extract

Conditional operator: ? to multiplex result

Example: (a==1)? funcif1 : funcif0

The above is equivalent to:

((a==1) && funcif1)

|| ((a!=1) && funcif0)

Registers and wires

Declaring a 4 bit wire with index starting at 0:

wire [3:0] w;

Declaring an 8 bit register:

reg [7:0] r;

Declaring a 32 element memory 8 bits wide:

reg [7:0] mem [0:31]

Bit extract example:

r[5:2] returns 4 bits between pos 2 to 5 inclusive

Assignment

Assignment to wires uses the assign primitive outside an always block, e.g.:

assign mywire = a & b

Registers are assigned to inside an always block which specifies where the clock comes from, e.g.:

always@(posedge myclock)

cnt = cnt + 1;

Blocking vs. unblocking assignment <= vs. =

The <= assignment operator is non-blocking (i.e. if use in an always@(posedge) it will be performed on every positive edge. If you have many non-blocking assignments they will all be updated in parallel. The <= operator must be used inside an always block – you can't use it in an assign statement.

The blocking assignment operator = can be used in either an assign block or an always block. But it causes assignments to be performed in sequential order. This tends to result in slower circuits, so avoid using it (especially for synthesized circuits) unless you have to.

Case and if statements

Case and if statements are used inside an always block to conditionally update state. e.g.:

```
always @(posedge clock)
  if (add1 && add2) r <= r+3;
  else if (add2) r <= r+2;
  else if (add1) r <= r+1;
```

Note that we don't need to specify what happens when add1 and add2 are both false since the default behavior is that r will not be updated. Equivalent function using a case statement:

```
always @(posedge clock)
  case({add2,add1})
    2'b11 : r <= r+3;
    2'b10 : r <= r+2;
    2'b01 : r <= r+1;
    default: r <= r;
  endcase
```

Module declarations

Modules pass inputs, outputs as wires by default.

```
module ModName (
  output reg [3:0] result, // register output
  input [1:0] bitsin, input clk, inout bidirectnl );
  ... code ...
endmodule
```

Verilog Simulation / ISIM commands

```
$display ("a string to display");
$monitor ("like printf. Vals: %d %b", decv,bitv);
#100 // wait 100ns or simulation moments
$finish // end simulation
```

Appendix B: ARM Assembly Language Cheatsheet

Memory access instructions

```
LDR Rd, [Rn]           ; load 32-bit number at [Rn] to Rd
LDR Rd, [Rn,#off]      ; load 32-bit number at [Rn+off] to Rd
STR Rt, [Rn]           ; store 32-bit Rt to [Rn]
STR Rt, [Rn,#off]      ; store 32-bit Rt to [Rn+off]
PUSH {Rt}              ; push 32-bit Rt onto stack
POP {Rd}               ; pop 32-bit number from stack into Rd

MOV{S} Rd, <op2>       ; set Rd equal to op2
MOV Rd, #im16           ; set Rd equal to im16, im16 is 0 to 65535
MVN{S} Rd, <op2>       ; set Rd equal to -op2
```

Branch instructions

```
B label                ; branch to label Always
BEQ label              ; branch if Z == 1 Equal
BNE label              ; branch if Z == 0 Not equal
BCS label              ; branch if C == 1 Higher or same, unsigned ≥
BHS label              ; branch if C == 1 Higher or same, unsigned ≥
BCC label              ; branch if C == 0 Lower, unsigned <
BLO label              ; branch if C == 0 Lower, unsigned <
BMI label              ; branch if N == 1 Negative
BPL label              ; branch if N == 0 Positive or zero
BVS label              ; branch if V == 1 Overflow
BVC label              ; branch if V == 0 No overflow
BHI label              ; branch if C==1 and Z==0 Higher, unsigned >
BLS label              ; branch if C==0 or Z==1 Lower or same, unsigned ≤
BGE label              ; branch if N == V Greater than or equal, signed ≥
BLT label              ; branch if N != V Less than, signed <
BGT label              ; branch if Z==0 and N==V Greater than, signed >
BLE label              ; branch if Z==1 or N!=V Less than or equal, signed ≤
BX Rm                  ; branch indirect to location specified by Rm
BL label               ; branch to subroutine at label
BLX Rm                 ; branch to subroutine indirect specified by Rm
```

Logical instructions

```
AND{S} {Rd}, Rn, <op2> ; Rd=Rn&op2 (op2 is 32 bits)
ORR{S} {Rd}, Rn, <op2> ; Rd=Rn|op2 (op2 is 32 bits)
EOR{S} {Rd}, Rn, <op2> ; Rd=Rn^op2 (op2 is 32 bits)
BIC{S} {Rd}, Rn, <op2> ; Rd=Rn&(~op2) (op2 is 32 bits)
ORN{S} {Rd}, Rn, <op2> ; Rd=Rn|(~op2) (op2 is 32 bits)
LSR{S} Rd, Rm, Rs      ; logical shift right Rd=Rm>>Rs (unsigned)
LSR{S} Rd, Rm, #n      ; logical shift right Rd=Rm>>n (unsigned)

ASR{S} Rd, Rm, Rs      ; arithmetic shift right Rd=Rm>>Rs (signed)
ASR{S} Rd, Rm, #n      ; arithmetic shift right Rd=Rm>>n (signed)
LSL{S} Rd, Rm, Rs      ; shift left Rd=Rm<<Rs (signed, unsigned)
LSL{S} Rd, Rm, #n      ; shift left Rd=Rm<<n (signed, unsigned)
```

Arithmetic instructions

```
ADD{S} {Rd}, Rn, <op2> ; Rd = Rn + op2
ADD{S} {Rd}, Rn, #im12 ; Rd = Rn + im12, im12 is 0 to 4095
SUB{S} {Rd}, Rn, <op2> ; Rd = Rn - op2
SUB{S} {Rd}, Rn, #im12 ; Rd = Rn - im12, im12 is 0 to 4095
RSB{S} {Rd}, Rn, <op2> ; Rd = op2 - Rn
RSB{S} {Rd}, Rn, #im12 ; Rd = im12 - Rn
CMP Rn, <op2>          ; Rn - op2 sets the NZVC bits
MUL{S} {Rd}, Rn, Rm     ; Rd = Rn * Rm signed or unsigned
```

Notes

Ra Rd Rm Rn Rt represent 32-bit registers

value any 32-bit value: signed, unsigned, or address

{S} if S is present, instruction will set condition codes

#im12 any value from 0 to 4095

#im16 any value from 0 to 65535

{Rd,} if Rd is present Rd is destination, otherwise Rn

#n any value from 0 to 31

#off any value from -255 to 4095

label any address within the ROM of the microcontroller

op2 the value generated by <op2>

EEE3096S Embedded Systems II

Final Exam 2025

MULTIPLE CHOICE ANSWER SHEET

Please detach this sheet, fill in your name and student number, include this page in your answer script.

Your Name:		Student Num:	
-------------------	--	---------------------	--

Fill in your answer choice using a cross (X). If you change your choice then scratch out () your initial choice to ensure your revised option is clearly indicated. Select **only one option unless** question explicitly states you can choose more than one answer option.

Question	Answer Choice (select option with an X)			
A.1	a)	b)	c) X	d)
A.2	a)	b) X	c)	d)
A.3	a)	b)	c) X	d)
A.4	a)	b)	c) X	d)
A.5	a)	b)	c)	d) X
A.6	a)	b) X	c)	d)
A.7	a)	b) X	c)	d)
A.8	a)	b) X	c)	d)

SECTION B3: please answer your True/False questions using this table by ticking a cell in either the True or False row to indicate your answer to the questions listed in the top row.

Question	B3.1	B3.2	B3.3	B3.4	B3.5
True				✓	✓
False	✓	✓	✓		